

QUIC Packet Receive Timestamps

draft-ietf-quic-receive-ts
QUIC @ IETF 126

Joseph Beshay (Meta), Ian Swett (Google)

Overview of Draft

Enable the peer to report packet receive timestamps for some or all ACKed packets. The timestamp is connection local and not intended to be absolute.

Uses new ACK frame types that extend existing ACK and PATH_ACK frames with receive timestamp information:

```
Timestamp Range {  
    Delta Largest Acknowledged (i),  
    Timestamp Delta Count (i),  
    Timestamp Delta (i) ...,  
}
```

draft-ietf-quic-receive-ts-02 → draft-ietf-quic-receive-ts-03

Editorial changes only:

- Clarify default for `max_receive_timestamps_per_ack` is 0 and prohibits frames
- Add security considerations for timestamp manipulation and resource exhaustion

1 Open Issue

Open Issue ([#43](#)): Do we need specific guidance on rejecting 'implausible' timestamps

Receive timestamps are peer-controlled (integrity-protected but not trusted) — peer can report time advancing slower/faster than real time (e.g., 1ms per 2ms) to inflate/deflate BWE

Affects delay-based CC / BWE — BBR can bound by send rate

Current draft: says SHOULD do plausibility checks, MAY ignore implausible timestamps — no concrete heuristics

Victor and Ian suggested two heuristics to detect implausible timestamps:

- Too large: delta vs first timestamp $\gg$ local elapsed
- Too small: delta vs first timestamp $\ll$ local elapsed - SRTT or PTO timer

Should we include guidance at all? Are there other heuristics we should include?

Implementation Status

- Mvfst supports the latest draft version.
 - Not enabled on the internet, but available in code and interop endpoints.
- Google Quiche has support for the latest format.
 - Transport param negotiation is planned.

What's next?

- What's left to Add/Fix/etc in the draft?
 - It seems like the right featureset to the authors, does the WG agree?
 - Please read for clarity and editorial suggestions.
- Any interest in interop testing?